

NATIONAL TECHNICAL UNIVERSITY OF ATHENS

SCHOOL OF ELECTRICAL AND COMPUTER ENGINEERING
DEPARTMENT OF COMPUTER SCIENCE AND ENGINEERING
COMPUTER SYSTEMS LABORATORY

<http://www.cslab.ece.ntua.gr>

Operating Systems

6th Semester, Academic Year 2024-2025

2nd Laboratory Exercise

Synchronization

Computer Systems Laboratory, NTUA
April 2025

In this laboratory exercise, you will use common synchronization mechanisms to solve synchronization problems in multithreaded applications based on the POSIX threads (pthreads) standard.

The mechanisms you will use are:

- **Mutexes, semaphores, and condition variables** as defined by POSIX.
- **Atomic operations**, as defined by hardware and exposed to the programmer through GCC compiler built-ins.

For more information on POSIX threads (besides the abundance of online resources):

- David R. Butenhof, *"Programming with POSIX Threads"* (Amazon link).

- The "System Interfaces" section of the POSIX.1-2008 standard (available as man pages; on Debian-based systems, install the `manpages-posix` and `manpages-posix-dev` packages).

For more details on atomic operations, their implementation in GCC, and their use in large projects like the Linux kernel:

- The "Built-in functions for atomic memory access" section in the GCC manual.
- The "atomic ops" section in the Linux kernel documentation.

1. Synchronization in Existing Code

Familiarize yourself with the use of POSIX threads by studying the provided example `pthread-test.c`

Task: Study and Extend `simplesync.c`

You are given the program `simplesync.c`, which works as follows:

1. Initializes a variable `val = 0`.
2. Creates two threads that run concurrently:
 - The first thread increments `val` by 1, `N` times.
 - The second thread decrements `val` by 1, `N` times.
 - The threads do **not** synchronize their execution.

Tasks:

1. Use the provided `Makefile` to compile and run the program. What do you observe? Why?
2. Study how two different executables (`simplesync-atomic`, `simplesync-mutex`) are generated from the same source file (`simplesync.c`).
3. Extend the code in `simplesync.c` so that the threads in `simplesync-mutex` synchronize using **POSIX mutexes**. Verify correct execution.
4. Extend the code in `simplesync.c` so that the threads in `simplesync-atomic` synchronize using **GCC atomic operations**. Verify correct execution.

Note: All code files for this exercise are located in `/home/oslab/code/sync`. **Study the provided code carefully before starting!**

Questions:

1. Use the `time(1)` command to measure the execution time of the executables. How does the execution time of the synchronized versions compare to the original unsynchronized program? Why?
 2. Which synchronization method is faster: atomic operations or POSIX mutexes? Why?
 3. To which processor instructions are GCC atomic operations translated in your target architecture? Use `gcc -S -g` to generate assembly code (with source line information, e.g., `.loc 1 63 0`). Check the `make` output for compilation details.
 4. To which instructions are POSIX mutex operations translated? Provide an example of `pthread_mutex_lock()` in assembly, as in the previous question.
-

2. Parallel Computation of the Mandelbrot Set

You are given a program (`mandel.c`) that computes and renders the Mandelbrot set in a terminal (Fig. 1).

The **Mandelbrot set** is defined as the set of complex numbers c for which the sequence $z_{n+1}=z_n^2+c$ remains bounded. A simple algorithm to plot it:

1. Map the drawing area to a region in the complex plane.
2. For each pixel, compute the corresponding complex number c and iterate $z_{n+1}=z_n^2+c$, $z_0=0$, until $|z_n|>2$ or a maximum iteration count is reached.
3. The iteration count determines the pixel's color.

The program relies on the library `mandel-lib.{h,c}`, which provides:

- `mandel_iterations_at_point()`: Computes the color of a point (x,y) .
- `set_xterm_color()`: Sets the terminal's text color.

The output is a block of characters (`x_chars` columns $\times$ `y_chars` rows). The program calls `compute_and_output_mandel_line()` iteratively to print the block line by line.

Task: Parallelize the Mandelbrot Computation

Extend `mandel.c` to distribute the computation across `NTHREADS` POSIX threads (e.g., 3). Work is distributed by rows:

- Thread i (where $i=0,1,\dots,NTHREADS-1$) handles rows $i, i+NTHREADS, i+2NTHREADS, \dots, i+NTHREADS, i+2NTHREADS, \dots$

Implement two versions of the program, synchronizing threads using:

1. **POSIX semaphores** (`<semaphore.h>`). See `sem_overview(7)`, `sem_wait(3)`, `sem_post(3)`.
2. **POSIX condition variables** (`<pthread.h>`). See `pthread_cond_wait(3)`, `pthread_cond_broadcast(3)`, `pthread_cond_signal(3)`.

The number of threads (`NTHREADS`) should be specified at runtime as a command-line argument.

Note: If the terminal retains incorrect text colors after program execution (e.g., purple text on black), use the `reset` command to restore defaults.

Questions:

1. How many semaphores are needed for your synchronization scheme?
2. Compare the execution time of the serial and parallel versions (with 2 threads). Use `time(1)` for measurements. Test on a machine with at least 2 CPU cores (check with `cat /proc/cpuinfo`).
3. How many condition variables did you use in the second version? If only one is used, how does synchronization work, and what performance issue arises?
4. Does the parallel program achieve speedup? If not, why? What is the problem with your synchronization scheme? *Hint:* How large is the critical section? Should it include both computation and output phases?
5. What happens to the terminal if you press Ctrl+C during execution? How could you modify `mandel.c` to ensure the terminal reverts to its previous state even if interrupted?

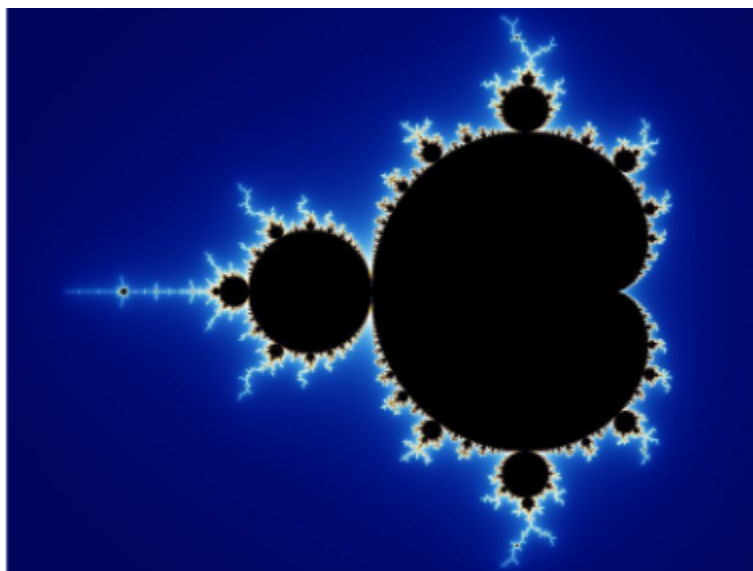


Fig. 1: